Python 之伊斯蘭幾何藝術

軟體需求：
Chrome 瀏覽器
以電子郵件帳號於 Repl.it 雲端平台, 免費註冊

Python 運行環境：
Python 3.6
Matplotlib
Shapely

2018-11-12 第一版

目錄

Assignments (學生自行研習)

S01_課程內容概要

S02_幾何藝術的背景知識

S03_Python 程式運算的方式

S04_幾何藝術的學術研究

S05_2D 仿射變換

Projects (學生實作)

A01-for-loop

A02-draw-circle-using-cos-sin

A03-繪製無邊框正方形, 底色為淺藍

A04-以右下角為圓心 R 為半徑, 繪製四分之一的紅色圓弧

A05-繪製黑色對角線

A06-求黑色對角線之交點

A07-求圓弧與線段的交點

A08-繪製內圓之綠色半徑線段

A09-繪製藍色內圓

A10-繪製右下角之 1/4 圓弧

A11-繪製右上、左上、左下圓弧-學生作答

A12-求兩點的座標並標示其位置

A13-繪製十字線段-學生作答

A14-用反射變換及滑動變換-將圖面填滿十字形

A15-完成的紋飾

A16-如何製作杯墊? (How to make a coaster?)

A17-Task-review

A01-for-loop

```
# 01.py
#  簡短的程式範例
#
# 2018-10-28
#
sum = 0
for i in (1,2,3,4,5,6,7,8,9,10) :
sum = sum + i
print('i=',i,'sum is', sum)
# end for
print('-------------------')
#
sum = 0
for i in range(1,11) :
sum = sum + i
print('i=',i,'sum is', sum)
# end for
print('-------------------')
#
"""
ASCII (American Standard Code for Information Interchange，美國資訊交換標準程式碼) 是基於拉丁字母的一套電腦編碼系統，它主要用於顯示現代英語，ASCII 最後一次更新則是在 1986 年，至今為止共定義了 128 個字元；其中 33 個字元無法顯示，且這 33 個字元多數都已是陳舊廢止的控制字元，在 33 個字元之外的是 95 個可顯示的字元。
"""
#
sum = ""
for i in range(ord('A'), ord('Z')) :
sum = sum + chr(i)
print('i=',i,'sum is', sum)
# end for
print('-------------------')
#
sum = ""
for i in range(ord('a'), ord('z')) :
```

```
sum = sum + chr(i)
print('i=',i,'sum is', sum)
# end for
print('----------------------------------------')
#
###
#
# !! for degree 90, 180, 270, 360
# !! very small value was represented by scientific expression
# !! ie: Cos(3*pi/2)= -1.8369701987210297e-16
#
import math
step = math.pi/180.0
#
for i in range(0, 361) :
print('i=', i,
'value is', i*step,
';',
'Cos(θ)=', math.cos(i*step),
',',
'Sin(θ)=', math.sin(i*step))
# end for
print('----------------------------------------')
#
# round() function sometimes will reulut in strange Value
# ie: -0.0
#
for i in range(0, 361) :
print('i=', i,
'value is', round(i*step,6),
';',
'Cos(θ)=', round(math.cos(i*step),6),
',',
'Sin(θ)=', round(math.sin(i*step),6))
# end for
```

A02-draw-circle-using-cos-sin

```
# 02-draw-circle-using-cos-sin.py
# 2018-10-28
# array  —  Efficient arrays of numeric values
# https://docs.python.org/3/library/array.html
# array  的程式範例
#
#  載入繪圖模組  pyplot,  重新命名為  plot
import matplotlib.pyplot as plot
import matplotlib.patches as patches
#
import math
import array
#
from matplotlib.ticker import MultipleLocator
#
# ----------------------------------------------------------------
#
def set_graphic_area(width,height) :

    cm2inch = 1/2.54       # inch per cm
    #
    # define graphic area
    #
    left_margin = 1.0     # cm
    right_margin = 1.0    # cm
    #
    figure_width    = width    # cm , from xmin to xmax
    figure_height = height # cm , from ymin to ymax
    #
    top_margin = 1.0       # cm
    bottom_margin = 1.0 # cm
    #
    box_width = left_margin + figure_width + right_margin     # cm
    box_height = top_margin + figure_height + bottom_margin # cm
    #
    top_value       = 1.0 - top_margin / box_height
```

```
    bottom_value = bottom_margin / box_height
    left_value      = left_margin / box_width
    right_value    = 1.0 - right_margin / box_width
    #
    return
(box_width*cm2inch,box_height*cm2inch,top_value,bottom_value,lef
t_value,right_value,width)
#
# end of def
#
# setup 圖面大小 (20, 20) 單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 10
tup7 = set_graphic_area(2*r0, 2*r0)
# 呼叫 pyplot.figure(), 建立一個圖表物件, 並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
# 圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                    top       = tup7[2] ,
                    bottom = tup7[3] ,
                    left      = tup7[4] ,
                    right    = tup7[5] ,
                    )
#
plot.xlim(-tup7[6]/2, tup7[6]/2)
# 設定 X 軸的顯示範圍, from xmin to xmax
plot.ylim(-tup7[6]/2, tup7[6]/2)
# 設定 Y 軸的顯示範圍, from ymin to ymax
#
# Set minor tick locations.
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
```

```
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
ax.set_axis_on()         # will turn on    grid line
# ax.set_axis_off()    # will turn off grid line
#
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
#
# ----------------------------------------------------------------
#
xs = array.array('f')
ys = array.array('f')
step = math.pi/180.0
#
r1 = 8
for i in range(0, 361) :
      # Cos(θ) = math.cos(i*step)
      # Sin(θ) = math.sin(i*step))
      xs.append(math.cos(i*step)*r1)
      ys.append(math.sin(i*step)*r1)
# end for
#
# print(type(xs))
# print(xs)
#
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=10)
#
print ('Done')
ax.set_title('02', fontsize=22)    #  設定子圖的標題
plot.savefig("02.png", format="png", dpi=72)
```

mylib.py

```
# 2018-11-12
# python3
# rep;.it web Python 3.61
#
import numpy as np
from shapely.geometry    import LineString
#
# specifying the width and the height of the box in inches
# declare graphics area
#
# here width == height
#  定義圖面函數  ----------------------------------------
def set_graphic_area(width,height) :

        cm2inch = 1/2.54        # inch per cm
        #
        # define graphic area
        #
        left_margin = 1.0      # cm
        right_margin = 1.0    # cm
        #
        figure_width    = width    # cm , from xmin to xmax
        figure_height = height # cm , from ymin to ymax
        #
        top_margin = 1.0        # cm
        bottom_margin = 1.0 # cm
        #
        box_width = left_margin + figure_width + right_margin      # cm
        box_height = top_margin + figure_height + bottom_margin # cm
        #
        top_value        = 1.0 - top_margin / box_height
        bottom_value = bottom_margin / box_height
        left_value      = left_margin / box_width
        right_value    = 1.0 - right_margin / box_width
```

```
    #
    return
(box_width*cm2inch,box_height*cm2inch,top_value,bottom_value,lef
t_value,right_value,width)
    #
# end of def
#
# Define the arc
# center position is cxy = (cx, cy)
# start_angle, end_angle is in degrees
#
def shapely_Arc(cxy, r, start_angle, end_angle) :
    #
    numsegments = 1000
    # The coordinates of the arc
    theta = np.radians(np.linspace(start_angle, end_angle,
numsegments))
    x = cxy[0] + r * np.cos(theta)
    y = cxy[1] + r * np.sin(theta)
    Arc = LineString(np.column_stack([x, y]))
    return Arc
    #
# end of def
#
```

A03-繪製無邊框正方形, 底色為淺藍

```
# 03.py
# 2018-10-29
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
#  載入繪圖模組  pyplot,  重新命名為  plot
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely.geometry import LineString
from matplotlib.ticker    import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小  (18,18)  單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫  pyplot.figure(),  建立一個圖表物件,  並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
```

```
# Set minor tick locations.
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_01.png, 撰寫程式碼, 將繪圖結果存放於 03.png
#  以圖面中心為原點:
# 1. 繪製無邊框正方形, 底色為淺藍
#
# square has four points, square one side length is R
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none',
facecolor=lightBlue, fill=True, zorder=10)
# ----------------------------------------------------------------
print ('Done')
ax.set_title('03', fontsize=22)   #  設定子圖的標題
plot.savefig("03.png", format="png", dpi=72)
```

A04-以右下角為圓心 R 為半徑, 繪製四分之一的紅色圓弧

```
# 04.py
# 2018-10-29
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
import matplotlib.pyplot as plot
from mylib                import set_graphic_area, shapely_Arc
from shapely.geometry import LineString
from matplotlib.ticker    import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小 (18,18) 單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫 pyplot.figure(), 建立一個圖表物件, 並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                    top       = tup7[2] ,
                    bottom = tup7[3] ,
                    left      = tup7[4] ,
                    right    = tup7[5] ,
                    )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
# Set minor tick locations.
```

```
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_01.png, 撰寫程式碼, 將繪圖結果存放於 04.png
#  以圖面中心為原點:
# 1. 繪製無邊框正方形, 底色為淺藍
# 2. 以正方形右下角為圓心, R 為半徑, 繪製四分之一的紅色圓弧, 起始於π/2, 終止於π
#
# square has four points, square one side length is R
# square center is (0,0)
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none',
facecolor=lightBlue, fill=True, zorder=10)
#
# shapely_Arc(cxy, r, start_angle, end_angle)
myArc = shapely_Arc(pA, R, 90, 180)
```

```
xs, ys = myArc.xy
ax.plot(xs, ys, color='red', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=10)
#
# ----------------------------------------------------------------
#
print ('Done')
ax.set_title('04', fontsize=22)    #  設定子圖的標題
plot.savefig("04.png", format="png", dpi=72)
```

A05-繪製黑色對角線

```
# 05.py
# 2018-10-29
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely.geometry    import LineString
from matplotlib.ticker      import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小  (18,18)  單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫  pyplot.figure(),  建立一個圖表物件,  並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
# Set minor tick locations.
```

```
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_01.png, 撰寫程式碼, 將繪圖結果存放於 05.png
#  以圖面中心為原點:
# 1. 繪製無邊框正方形, 底色為淺藍
# 2. 以正方形右下角為圓心, R 為半徑, 繪製四分之一的紅色圓弧, 起始於π/2, 終止於π
# 3. 繪製黑色對角線
#
# square has four points, square one side length is R
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none',
facecolor=lightBlue, fill=True, zorder=10)
#
# shapely_Arc(cxy, r, start_angle, end_angle)
myArc = shapely_Arc(pA, R, 90, 180)
```

```
xs, ys = myArc.xy
ax.plot(xs, ys, color='red', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=10)
#
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
for line in (lineA, lineB) :
	xs, ys = line.xy
	ax.plot(xs, ys, color='black', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=10)
# end for
#
# ------------------------------------------------------------
#
print ('Done')
ax.set_title('05', fontsize=22)	#  設定子圖的標題
plot.savefig("05.png", format="png", dpi=72)
```

A06-求黑色對角線之交點

```
# 06.py
# 2018-10-29
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely.geometry    import LineString
from matplotlib.ticker      import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小  (18,18)  單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫  pyplot.figure(),  建立一個圖表物件,  並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
```

```
# Set minor tick locations.
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_01.png, 撰寫程式碼, 將繪圖結果存放於 06.png
#  以圖面中心為原點:
# 1. 繪製無邊框正方形, 底色為淺藍
# 2. 以正方形右下角為圓心 R 為半徑, 繪製四分之一的紅色圓弧, 起始於π/2, 終止於π
# 3. 繪製黑色對角線
# 4. 求對角線的交點(blue)
#
# square has four points, square one side length is R
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none', facecolor=lightBlue, fill=True, zorder=10)
#
# shapely_Arc(cxy, r, start_angle, end_angle)
```

```
myArc = shapely_Arc(pA, R, 90, 180)
xs, ys = myArc.xy
ax.plot(xs, ys, color='red', alpha=0.7, linewidth=2.5, solid_capstyle='round', zorder=10)
#
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
for line in (lineA, lineB) :
    xs, ys = line.xy
    ax.plot(xs, ys, color='black', alpha=0.7, linewidth=2.5, solid_capstyle='round', zorder=10)
# end for
#
its1 = lineA.intersection(lineB)
its1_Arc = shapely_Arc((its1.x, its1.y), 0.1, 0, 360)
xs, ys = its1_Arc.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=2.5, solid_capstyle='round', zorder=20)
#
# ----------------------------------------------------------------
#
print ('Done')
ax.set_title('06', fontsize=22)   #  設定子圖的標題
plot.savefig("06.png", format="png", dpi=72)
```

A07-求圓弧與線段的交點

```
# 07-Arc-Line-intersection.py
# 2018-10-29
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely.geometry   import LineString
from matplotlib.ticker     import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小  (18,18)  單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫  pyplot.figure(),  建立一個圖表物件,  並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
# Set minor tick locations.
```

```
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_01.png, 撰寫程式碼, 將繪圖結果存放於 07.png
#  以圖面中心為原點:
# 1. 繪製無邊框正方形, 底色為淺藍
# 2. 以正方形右下角為圓心 R 為半徑, 繪製四分之一的紅色圓弧, 起始於π/2, 終止於π
# 3. 繪製黑色對角線
# 4. 求對角線的交點(藍色)
# 5. 求 red 圓弧與對角線的交點(blue)
#
# square has four points, square one side length is R
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none', facecolor=lightBlue, fill=True, zorder=10)
#
```

```
# shapely_Arc(cxy, r, start_angle, end_angle)
myArc = shapely_Arc(pA, R, 90, 180)
xs, ys = myArc.xy
ax.plot(xs, ys, color='red', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=10)
#
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
for line in (lineA, lineB) :
    xs, ys = line.xy
    ax.plot(xs, ys, color='black', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=10)
# end for
#
its1 = lineA.intersection(lineB)
its2 = lineA.intersection(myArc)
#
its1_Arc = shapely_Arc((its1.x, its1.y), 0.1, 0, 360)
xs, ys = its1_Arc.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
its2_Arc = shapely_Arc((its2.x, its2.y), 0.1, 0, 360)
xs, ys = its2_Arc.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
# ----------------------------------------------------------------
#
print ('Done')
ax.set_title('07', fontsize=22)   #  設定子圖的標題
plot.savefig("07.png", format="png", dpi=72)
```

A08-繪製內圓之綠色半徑線段

```
# 08.py
# 2018-10-30
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely.geometry    import LineString
from matplotlib.ticker       import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小  (18,18)  單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫  pyplot.figure(),  建立一個圖表物件,  並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
# Set minor tick locations.
```

```
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_01.png, 撰寫程式碼, 將繪圖結果存放於 08.png
#  以圖面中心為原點:
# 1. 繪製無邊框正方形, 底色為淺藍
# 2. 以正方形右下角為圓心 R 為半徑, 繪製四分之一的紅色圓弧, 起始於π/2, 終止於π
# 3. 繪製黑色對角線
# 4. 求對角線的交點(藍色)
# 5. 求綠色圓弧與對角線的交點(藍色)
# 6. 以綠線連接藍藍兩點
#
# square has four points, square one side length is R
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none', facecolor=lightBlue, fill=True, zorder=10)
```

```
#
# shapely_Arc(cxy, r, start_angle, end_angle)
myArc = shapely_Arc(pA, R, 90, 180)
xs, ys = myArc.xy
ax.plot(xs, ys, color='red', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=10)
#
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
for line in (lineA, lineB) :
    xs, ys = line.xy
    ax.plot(xs, ys, color='black', alpha=0.7, linewidth=1.5,
solid_capstyle='round', zorder=10)
# end for
#
its1 = lineA.intersection(lineB)
its2 = lineA.intersection(myArc)
#
its1_Arc = shapely_Arc((its1.x, its1.y), 0.1, 0, 360)
xs, ys = its1_Arc.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
its2_Arc = shapely_Arc((its2.x, its2.y), 0.1, 0, 360)
xs, ys = its2_Arc.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
lineC = LineString([(its1.x, its1.y), (its2.x, its2.y)])
xs, ys = lineC.xy
ax.plot(xs, ys, color='green', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
r1 = its1.distance(its2)
print('r1 =', r1) # we need r1 in next program
#
# ----------------------------------------------------------------
```

```
#
print ('Done')
ax.set_title('08', fontsize=22)    #  設定子圖的標題
plot.savefig("08.png", format="png", dpi=72)
```

A09-繪製藍色內圓

```
# 09-draw-Arc360.py
# 2018-10-30
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely.geometry    import LineString
from matplotlib.ticker      import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小  (18,18)  單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫  pyplot.figure(),  建立一個圖表物件,  並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
# Set minor tick locations.
```

```
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_01.png, 撰寫程式碼, 將繪圖結果存放於 09.png
#  以圖面中心為原點:
# 1. 繪製無邊框正方形, 底色為淺藍
# 2. 以正方形右下角為圓心 R 為半徑, 繪製四分之一的紅色圓弧, 起始於π
/2, 終止於π
# 3. 繪製黑色對角線
# 4. 求對角線的交點(藍色)
# 5. 求綠色圓弧與對角線的交點(藍色)
# 6. 以綠線連接藍藍兩點
# 7. draw blue circle
#
# square has four points, square one side length is R
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
```

```
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none',
facecolor=lightBlue, fill=True, zorder=10)
#
# shapely_Arc(cxy, r, start_angle, end_angle)
myArc = shapely_Arc(pA, R, 90, 180)
xs, ys = myArc.xy
ax.plot(xs, ys, color='red', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=10)
#
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
for line in (lineA, lineB) :
    xs, ys = line.xy
    ax.plot(xs, ys, color='black', alpha=0.7, linewidth=1.5,
solid_capstyle='round', zorder=10)
# end for
#
its1 = lineA.intersection(lineB)
its2 = lineA.intersection(myArc)
#
its1_Arc = shapely_Arc((its1.x, its1.y), 0.1, 0, 360)
xs, ys = its1_Arc.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
its2_Arc = shapely_Arc((its2.x, its2.y), 0.1, 0, 360)
xs, ys = its2_Arc.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
lineC = LineString([(its1.x, its1.y), (its2.x, its2.y)])
xs, ys = lineC.xy
ax.plot(xs, ys, color='green', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
r1 = its1.distance(its2)
print('r1 =', r1) # we need r1 in next program
```

```
#
Arc360 = shapely_Arc((its1.x, its1.y), r1, 0, 360)
xs, ys = Arc360.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=1.5, solid_capstyle='round', zorder=10)
#
# ----------------------------------------------------------------
#
print ('Done')
ax.set_title('09', fontsize=22)   #  設定子圖的標題
plot.savefig("09.png", format="png", dpi=72)
```

A10-繪製右下角之 1/4 圓弧

```
# 10-draw-ArcA.py
# 2018-10-30
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely.geometry    import LineString
from matplotlib.ticker      import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小  (18,18)  單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫  pyplot.figure(),  建立一個圖表物件,  並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
# Set minor tick locations.
```

```
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_01.png, 撰寫程式碼, 將繪圖結果存放於 10.png
#  以圖面中心為原點:
# 1.  繪製無邊框正方形, 底色為淺藍
# 2.  以正方形右下角為圓心 R 為半徑, 繪製四分之一的紅色圓弧, 起始於π/2, 終止於π
# 3.  繪製黑色對角線
# 4.  求對角線的交點(藍色)
# 5.  求綠色圓弧與對角線的交點(藍色)
# 6.  以綠線連接藍藍兩點
# 7. draw blue circle
# 8. draw ArcA at (Right, Down) corner
#
# square has four points, square one side length is R
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
```

```
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none',
facecolor=lightBlue, fill=True, zorder=10)
#
# shapely_Arc(cxy, r, start_angle, end_angle)
myArc = shapely_Arc(pA, R, 90, 180)
xs, ys = myArc.xy
ax.plot(xs, ys, color='red', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=10)
#
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
for line in (lineA, lineB) :
    xs, ys = line.xy
    ax.plot(xs, ys, color='black', alpha=0.7, linewidth=1.5,
solid_capstyle='round', zorder=10)
# end for
#
its1 = lineA.intersection(lineB)
its2 = lineA.intersection(myArc)
#
its1_Arc = shapely_Arc((its1.x, its1.y), 0.1, 0, 360)
xs, ys = its1_Arc.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
its2_Arc = shapely_Arc((its2.x, its2.y), 0.1, 0, 360)
xs, ys = its2_Arc.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
lineC = LineString([(its1.x, its1.y), (its2.x, its2.y)])
xs, ys = lineC.xy
ax.plot(xs, ys, color='green', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
r1 = its1.distance(its2)
print('r1 =', r1) # we need r1 in next program
```

```
#
Arc360 = shapely_Arc((its1.x, its1.y), r1, 0, 360)
xs, ys = Arc360.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=1.5, solid_capstyle='round', zorder=10)
#
ArcA = shapely_Arc(pA, r1, 90, 180)
xs, ys = ArcA.xy
ax.plot(xs, ys, color='blue', alpha=0.7, linewidth=2.5, solid_capstyle='round', zorder=20)
"""
ArcB = #  用正確的程式碼取代  this line
ArcC = #  用正確的程式碼取代  this line
ArcD = #  用正確的程式碼取代  this line
ArcSet = (ArcA, ArcB, ArcC, ArcD)
for Arc in ArcSet :
    xs, ys = Arc.xy
    ax.plot(xs, ys, color='blue', alpha=0.7, linewidth=2.5, solid_capstyle='round', zorder=10)
# end for
"""
#
# ------------------------------------------------------------
#
print ('Done')
ax.set_title('10', fontsize=22)   #  設定子圖的標題
plot.savefig("10.png", format="png", dpi=72)
```

A11-繪製右上、左上、左下圓弧-學生作答

```
# 11-draw-ArcB-ArcC-ArcD-student.py
# 2018-10-30
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely.geometry    import LineString
from matplotlib.ticker      import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小  (18,18)  單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫  pyplot.figure(),  建立一個圖表物件,  並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
# Set minor tick locations.
```

```
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_01.png, 撰寫程式碼, 將繪圖結果存放於 11.png
#  以圖面中心為原點:
# 1.  繪製無邊框正方形, 底色為淺藍
# 2.  以正方形右下角為圓心 R 為半徑, 繪製四分之一的紅色圓弧, 起始於π
/2, 終止於π
# 3.  繪製黑色對角線
# 4.  求對角線的交點(藍色)
# 5.  求綠色圓弧與對角線的交點(藍色)
# 6.  以綠線連接藍藍兩點
# 7. draw blue circle
# 8. draw ArcA at (Right, Down) corner
# 9. draw ArcB at (Right, Up    ) corner
# .. draw ArcC at (Left,    Up    ) corner
# .. draw ArcD at (Left,    Down) corner
#
# square has four points, square one side length is R
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# combine these four poins to make LineString()
```

```
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none',
facecolor=lightBlue, fill=True, zorder=10)
#
# shapely_Arc(cxy, r, start_angle, end_angle)
myArc = shapely_Arc(pA, R, 90, 180)
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
#
its1 = lineA.intersection(lineB)
its2 = lineA.intersection(myArc)
lineC = LineString([(its1.x, its1.y), (its2.x, its2.y)])
xs, ys = lineC.xy
ax.plot(xs, ys, color='green', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
r1 = its1.distance(its2)
print('r1 =', r1) # we need r1 in next program
#
Arc360 = shapely_Arc((its1.x, its1.y), r1, 0, 360)
xs, ys = Arc360.xy
ax.plot(xs, ys, color='blue', alpha=1.0, linewidth=1.5,
solid_capstyle='round', zorder=10)
#
ArcA = shapely_Arc(pA, r1, 90, 180)
xs, ys = ArcA.xy
ax.plot(xs, ys, color='blue', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
# 將你的程式碼放在這個區塊
#

"""
ArcB = # 用正確的程式碼取代 this line
ArcC = # 用正確的程式碼取代 this line
```

```
ArcD = #  用正確的程式碼取代  this line
ArcSet = (ArcA, ArcB, ArcC, ArcD)
for Arc in ArcSet :
	xs, ys = Arc.xy
	ax.plot(xs, ys, color='blue', alpha=0.7, linewidth=2.5, solid_capstyle='round', zorder=10)
# end for
"""

#
# ----------------------------------------------------------------
#
print ('Done')
ax.set_title('11', fontsize=22)   #  設定子圖的標題
plot.savefig("11.png", format="png", dpi=72)
```

A12-求兩點的座標並標示其位置

```
# 12-draw-two-point.py
# 2018-10-30
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
#  要執行本程式, 請按上方 run▶
#
import matplotlib.pyplot as plot
from mylib import set_graphic_area, shapely_Arc
from shapely import affinity
from shapely.geometry    import Point, LineString
from matplotlib.ticker import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小 (18,18) 單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫 pyplot.figure(), 建立一個圖表物件, 並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
```

```
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
# Set minor tick locations.
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_02.png, 撰寫程式碼, 將繪圖結果存放於 12.png
#  以圖面中心為原點:
# a. find p1 Point position
# b. draw p1 as red Point
# c. find p2 Point position
# d. draw p2 as green Point
#
# square has four points, square one side length is R
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none',
facecolor=lightBlue, fill=True, zorder=10)
```

```
#
# shapely_Arc(cxy, r, start_angle, end_angle)
myArc = shapely_Arc(pA, R, 90, 180)
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
#
its1 = lineA.intersection(lineB)
its2 = lineA.intersection(myArc)
r1 = its1.distance(its2)
print('r1 =', r1) # we need r1 in next program
#
ArcA = shapely_Arc(pA, r1,   90, 180)
ArcB = shapely_Arc(pB, r1, 180, 270)
ArcC = shapely_Arc(pC, r1, 270, 360)
ArcD = shapely_Arc(pD, r1,     0,   90)
ArcSet = (ArcA, ArcB, ArcC, ArcD)
for Arc in ArcSet :
    xs, ys = Arc.xy
    ax.plot(xs, ys, color='blue', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=10)
# end for
#
# 已知 R, r1, and (0,0)在圖面中心
# 求 red p1 & green p2 Point 座標?
p1 = (R/2-r1, -R/2)
Point1 = Point(p1)
Point2 = affinity.rotate(Point1, 180, (0,0))
p2 = (Point2.x, Point2.y)
#
p1_Arc360 = shapely_Arc(p1, 0.1,   0, 360)
p2_Arc360 = shapely_Arc(p2, 0.1,   0, 360)
#
xs, ys = p1_Arc360.xy
ax.plot(xs, ys, color='red', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
xs, ys = p2_Arc360.xy
```

```
ax.plot(xs, ys, color='green', alpha=1.0, linewidth=2.5, solid_capstyle='round', zorder=20)
#
# ----------------------------------------------------------------
#
print ('Done')
ax.set_title('12', fontsize=22)   #  設定子圖的標題
plot.savefig("12.png", format="png", dpi=72)
```

A13-繪製十字線段-學生作答

```
# 13-draw-cross-line.py
# 2018-10-30
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely                  import affinity
from shapely.geometry   import Point, LineString
from matplotlib.ticker     import MultipleLocator
#
# ---------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小 (18,18) 單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫 pyplot.figure(), 建立一個圖表物件, 並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
```

```
# Set minor tick locations.
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_02.png, 撰寫程式碼, 將繪圖結果存放於 13.png
#  以圖面中心為原點:
# a. find p1 Point position
# b. draw p1 as red Point
# c. find p2 Point position
# d. draw p2 as green Point
#
# square has four points, square one side length is R
R = 12
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none',
facecolor=lightBlue, fill=True, zorder=10)
#
# shapely_Arc(cxy, r, start_angle, end_angle)
```

```
myArc = shapely_Arc(pA, R, 90, 180)
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
#
its1 = lineA.intersection(lineB)
its2 = lineA.intersection(myArc)
r1 = its1.distance(its2)
print('r1 =', r1) # we need r1 in next program
#
ArcA = shapely_Arc(pA, r1,   90, 180)
ArcB = shapely_Arc(pB, r1, 180, 270)
ArcC = shapely_Arc(pC, r1, 270, 360)
ArcD = shapely_Arc(pD, r1,     0,   90)
ArcSet = (ArcA, ArcB, ArcC, ArcD)
for Arc in ArcSet :
    xs, ys = Arc.xy
    ax.plot(xs, ys, color='blue', alpha=0.7, linewidth=2.5,
solid_capstyle='round', zorder=10)
# end for
#
# 已知 R, r1, and (0,0)在圖面中心
# 求 red p1 & green p2 Point 座標?
#
p1 = (R/2-r1, -R/2)
Point1 = Point(p1)
Point2 = affinity.rotate(Point1, 180, (0,0))
p2 = (Point2.x, Point2.y)
#
p1_Arc360 = shapely_Arc(p1, 0.1,   0, 360)
p2_Arc360 = shapely_Arc(p2, 0.1,   0, 360)
#
xs, ys = p1_Arc360.xy
ax.plot(xs, ys, color='red', alpha=1.0, linewidth=2.5,
solid_capstyle='round', zorder=20)
#
xs, ys = p2_Arc360.xy
```

```
ax.plot(xs, ys, color='green', alpha=1.0, linewidth=2.5, solid_capstyle='round', zorder=20)
#
# ----------------------------------------------------------------
#  將你的程式碼放在這個區塊
#  提示: 使用旋轉變換
# affinity.rotate(geom, degree, (0,0))


"""
#
line1 = LineString([p1, p2])
xs, ys = line1.xy
ax.plot(xs, ys, alpha=1.0, color='green', linewidth=2.0, solid_capstyle='round', zorder=30)
#
line2 = #  用正確的程式碼取代  this line
xs, ys = line2.xy
ax.plot(xs, ys, alpha=1.0, color='green', linewidth=2.0, solid_capstyle='round', zorder=30)
#
"""
#
# ----------------------------------------------------------------
#
print ('Done')
ax.set_title('13', fontsize=22)   #  設定子圖的標題
plot.savefig("13.png", format="png", dpi=72)
```

A14-用反射變換及滑動變換-將圖面填滿十字形

```
# 14-reflect-cross-line.py
# 2018-10-30
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
#  要執行本程式, 請按上方 run▶
#
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely                  import affinity
from shapely.geometry   import Point, LineString
from matplotlib.ticker     import MultipleLocator
#
# ------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小 (18,18) 單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫 pyplot.figure(), 建立一個圖表物件, 並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
```

```
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
# Set minor tick locations.
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
ax.set_axis_on() # will turn on grid line
# ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_02.png, 撰寫程式碼, 將繪圖結果存放於 14.png
#  以圖面中心為原點:
# a. find p1 Point position
# b. draw p1 as red Point
# c. find p2 Point position
# d. draw p2 as green Point
#
# square has four points, square one side length is R
R = 4
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# combine these four poins to make LineString()
square = LineString([pA, pB, pC, pD])
xs, ys = square.xy
lightBlue = (229/255, 245/255, 237/255)
ax.fill(xs, ys, alpha=1.0, closed=True, edgecolor='none',
facecolor=lightBlue, fill=True, zorder=10)
```

```
#
# shapely_Arc(cxy, r, start_angle, end_angle)
myArc = shapely_Arc(pA, R, 90, 180)
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
#
its1 = lineA.intersection(lineB)
its2 = lineA.intersection(myArc)
r1 = its1.distance(its2)
print('r1 =', r1) # we need r1 in next program
#
#  已知  R, r1, and (0,0)在圖面中心
#  求  red p1 & green p2 Point  座標?
#
p1 = (R/2-r1, -R/2)
Point1 = Point(p1)
Point2 = affinity.rotate(Point1, 180, (0,0))
p2 = (Point2.x, Point2.y)
#
line1 = LineString([p1, p2])
line2 = affinity.rotate(line1, 90, (0,0))
for line in (line1, line2) :
    xs, ys = line.xy
    ax.plot(xs, ys, alpha=1.0, color='green', linewidth=2.5,
solid_capstyle='round', zorder=30)
#
# ---------------------------------------------
# define  反射變換  matrix for shapely
reflect_about_Xaxis    = [ 1, 0, 0,-1, 0,0]
reflect_about_Yaxis    = [-1, 0, 0, 1, 0,0]
reflect_about_Origin = [-1, 0, 0,-1, 0,0]
#
translate_X_P0R = [1, 0, 0, 1, 0*R, 0]
#
reflect_about_Yaxis_P1R = [-1, 0, 0, 1, 1*R, 0]
reflect_about_Yaxis_N1R = [-1, 0, 0, 1,-1*R, 0]
#
```

```
translate_X_P2R = [1, 0, 0, 1, 2*R, 0]
translate_X_N2R = [1, 0, 0, 1,-2*R, 0]
#
reflect_about_Xaxis_P1R = [-1, 0, 0, 1, 0, 1*R]
reflect_about_Xaxis_N1R = [-1, 0, 0, 1, 0,-1*R]
#
translate_Y_P2R = [1, 0, 0, 1, 0, 2*R]
translate_Y_N2R = [1, 0, 0, 1, 0,-2*R]
#
# ------------------------------------------
#  將你的程式碼放在這個區塊
#  提示:  使用反射變換
#
for line in (line1, line2) :
    for affine in (translate_X_P0R,
                   reflect_about_Yaxis_P1R,
                   reflect_about_Yaxis_N1R,
                   translate_X_P2R,
                   translate_X_N2R) :
        line_affine = affinity.affine_transform(line, affine)
        xs, ys = line_affine.xy
        ax.plot(xs, ys, alpha=1.0, color='green', linewidth=2.5,
solid_capstyle='round', zorder=30)
    # end for
# end for
#
ax.set_title('14A', fontsize=22)   #  設定子圖的標題
plot.savefig("14A.png", format="png", dpi=72)
#
for line in (line1, line2) :
    for affine in (translate_X_P0R,
                   reflect_about_Yaxis_P1R,
                   reflect_about_Yaxis_N1R,
                   translate_X_P2R,
                   translate_X_N2R) :
        line_affine = affinity.affine_transform(line, affine)
        xs, ys = line_affine.xy
```

```
            ax.plot(xs, ys, alpha=1.0, color='green', linewidth=2.5, solid_capstyle='round', zorder=30)
            for t in (reflect_about_Xaxis_P1R, reflect_about_Xaxis_N1R) :
                line_affine_t = affinity.affine_transform(line_affine, t)
                xs, ys = line_affine_t.xy
                ax.plot(xs, ys, alpha=1.0, color='Blue', linewidth=2.5, solid_capstyle='round', zorder=30)
            # end for
        # end for
# end for
#
ax.set_title('14B', fontsize=22)   #  設定子圖的標題
plot.savefig("14B.png", format="png", dpi=72)
#
for line in (line1, line2) :
        for affine in (translate_X_P0R,
                       reflect_about_Yaxis_P1R,
                       reflect_about_Yaxis_N1R,
                       translate_X_P2R,
                       translate_X_N2R) :
            line_affine = affinity.affine_transform(line, affine)
            xs, ys = line_affine.xy
            ax.plot(xs, ys, alpha=1.0, color='green', linewidth=2.5, solid_capstyle='round', zorder=30)
            for t1 in (reflect_about_Xaxis_P1R, reflect_about_Xaxis_N1R) :
                line_affine_t1 = affinity.affine_transform(line_affine, t1)
                xs, ys = line_affine_t1.xy
                ax.plot(xs, ys, alpha=1.0, color='Blue', linewidth=2.5, solid_capstyle='round', zorder=30)
            # end for
            for t2 in (translate_Y_P2R, translate_Y_N2R) :
                line_affine_t2 = affinity.affine_transform(line_affine, t2)
                xs, ys = line_affine_t2.xy
                ax.plot(xs, ys, alpha=1.0, color='red', linewidth=2.5, solid_capstyle='round', zorder=30)
            # end for
        # end for
```

```
# end for
#
ax.set_title('14C', fontsize=22)    #  設定子圖的標題
plot.savefig("14C.png", format="png", dpi=72)
#
print ('Done')
```

A15-完成的紋飾

```
# 15-reflect-cross-line.py
# 2018-10-30
# python3
# run on https://repl.it
# run on ubuntu 16.04 LTS
#
import matplotlib.pyplot as plot
from mylib                    import set_graphic_area, shapely_Arc
from shapely                  import affinity
from shapely.geometry   import Point, LineString
from matplotlib.ticker     import MultipleLocator
#
# ---------------------------------------------------------------
#  設定圖面大小
#
# setup 圖面大小 (18,18) 單位公分
# tup7 is type of Tuple, have 7 indexed value
#
r0 = 9
tup7 = set_graphic_area(2*r0, 2*r0)
#  呼叫 pyplot.figure(), 建立一個圖表物件, 並成為目前圖表物件
fig = plot.figure(figsize=(tup7[0], tup7[1]))
#  圖表的繪圖區域被分為 1 個子圖, 1 row, 1 column, plot to index 1
ax = fig.add_subplot(1,1,1)
fig.subplots_adjust(
                        top       = tup7[2] ,
                        bottom = tup7[3] ,
                        left      = tup7[4] ,
                        right    = tup7[5] ,
                        )
#
#  設定 X 軸的顯示範圍, from xmin to xmax
plot.xlim(-tup7[6]/2, tup7[6]/2)
#  設定 Y 軸的顯示範圍, from ymin to ymax
plot.ylim(-tup7[6]/2, tup7[6]/2)
#
```

```
# Set minor tick locations.
spacing = 1.0
minorLocator = MultipleLocator(spacing)
ax.xaxis.set_minor_locator(minorLocator)
ax.yaxis.set_minor_locator(minorLocator)
# Set grid to use minor tick locations.
ax.grid(which='minor')
ax.set_xticks([])
ax.set_yticks([])
# get current axes, set X,Y same ratio & scale
plot.gca().set_aspect('equal', adjustable='box')
# ax.set_axis_on() # will turn on grid line
ax.set_axis_off() # will turn off grid line
# ----------------------------------------------------------------
# New code area
#  任務描述: 參考 task_02.png, 撰寫程式碼, 將繪圖結果存放於 15.png
#  以圖面中心為原點:
# a. find p1 Point position
# b. draw p1 as red Point
# c. find p2 Point position
# d. draw p2 as green Point
#
# square has four points, square one side length is R
R = 4
pA = ( R/2,-R/2) #  右下
pB = ( R/2, R/2) #  右上
pC = (-R/2, R/2) #  左上
pD = (-R/2,-R/2) #  左下
#
# shapely_Arc(cxy, r, start_angle, end_angle)
myArc = shapely_Arc(pA, R, 90, 180)
lineA = LineString([pA, pC])
lineB = LineString([pB, pD])
#
its1 = lineA.intersection(lineB)
its2 = lineA.intersection(myArc)
r1 = its1.distance(its2)
```

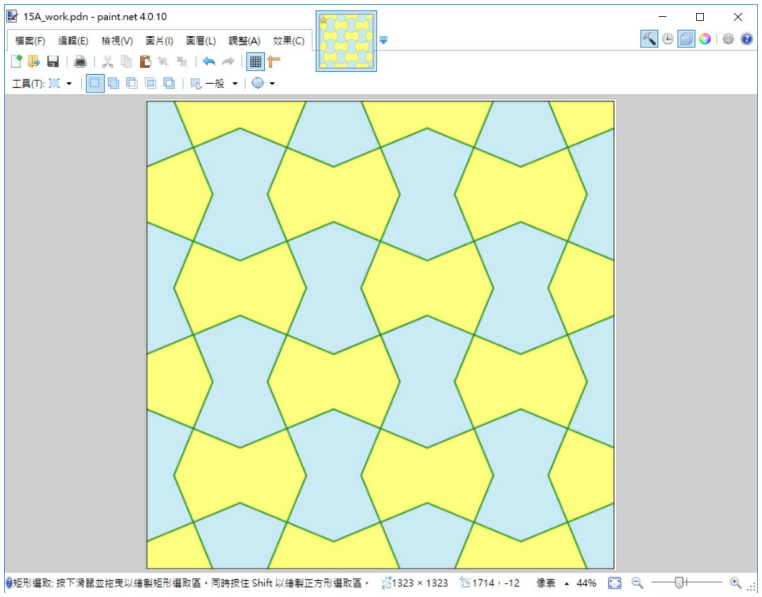
```
print('r1 =', r1) # we need r1 in next program
#
#  已知  R, r1, and (0,0)在圖面中心
#  求  red p1 & green p2 Point  座標?
#
p1 = (R/2-r1, -R/2)
Point1 = Point(p1)
Point2 = affinity.rotate(Point1, 180, (0,0))
p2 = (Point2.x, Point2.y)
#
line1 = LineString([p1, p2])
line2 = affinity.rotate(line1, 90, (0,0))
for line in (line1, line2) :
    xs, ys = line.xy
    ax.plot(xs, ys, alpha=1.0, color='green', linewidth=2.5,
solid_capstyle='round', zorder=30)
#
# ---------------------------------------------
# define  反射變換  matrix for shapely
reflect_about_Xaxis    = [ 1, 0, 0,-1, 0,0]
reflect_about_Yaxis    = [-1, 0, 0, 1, 0,0]
reflect_about_Origin = [-1, 0, 0,-1, 0,0]
#
translate_X_P0R = [1, 0, 0, 1, 0*R, 0]
#
reflect_about_Yaxis_P1R = [-1, 0, 0, 1, 1*R, 0]
reflect_about_Yaxis_N1R = [-1, 0, 0, 1,-1*R, 0]
#
translate_X_P2R = [1, 0, 0, 1, 2*R, 0]
translate_X_N2R = [1, 0, 0, 1,-2*R, 0]
#
reflect_about_Xaxis_P1R = [-1, 0, 0, 1, 0, 1*R]
reflect_about_Xaxis_N1R = [-1, 0, 0, 1, 0,-1*R]
#
translate_Y_P2R = [1, 0, 0, 1, 0, 2*R]
translate_Y_N2R = [1, 0, 0, 1, 0,-2*R]
#
```

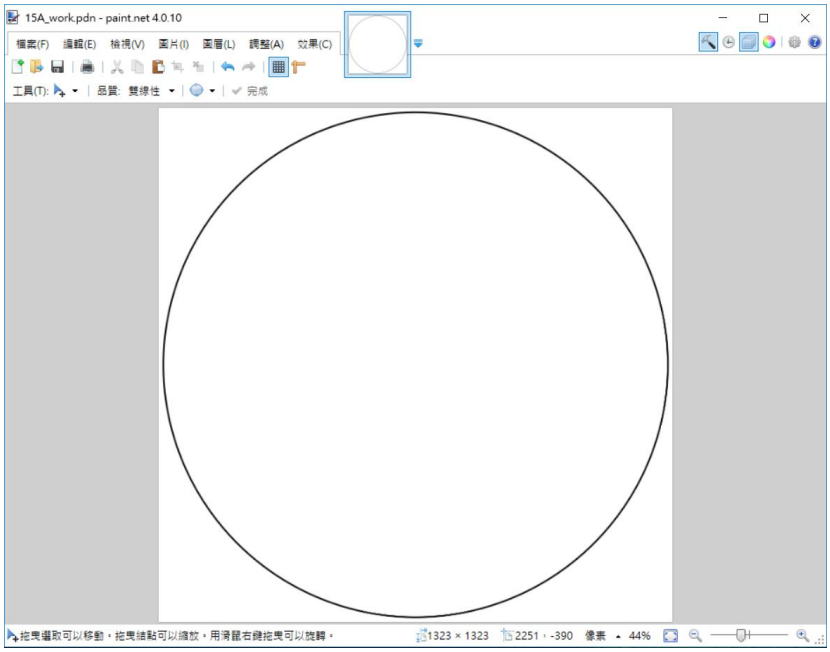
```
# ------------------------------------------
#  將你的程式碼放在這個區塊
#  提示: 使用反射變換
#
for line in (line1, line2) :
    for affine in (translate_X_P0R,
                   reflect_about_Yaxis_P1R,
                   reflect_about_Yaxis_N1R,
                   translate_X_P2R,
                   translate_X_N2R) :
        line_affine = affinity.affine_transform(line, affine)
        xs, ys = line_affine.xy
        ax.plot(xs, ys, alpha=1.0, color='green', linewidth=2.5, solid_capstyle='round', zorder=30)
        for t1 in (reflect_about_Xaxis_P1R, reflect_about_Xaxis_N1R) :
            line_affine_t1 = affinity.affine_transform(line_affine, t1)
            xs, ys = line_affine_t1.xy
            ax.plot(xs, ys, alpha=1.0, color='green', linewidth=2.5, solid_capstyle='round', zorder=30)
        # end for
        for t2 in (translate_Y_P2R, translate_Y_N2R) :
            line_affine_t2 = affinity.affine_transform(line_affine, t2)
            xs, ys = line_affine_t2.xy
            ax.plot(xs, ys, alpha=1.0, color='green', linewidth=2.5, solid_capstyle='round', zorder=30)
        # end for
    # end for
# end for
#
# square has four points, square one side length is R
R = 3.61
pA = (  R/2+2*R, -(R/2+2*R)) #  右下
pB = (  R/2+2*R,  (R/2+2*R)) #  右上
pC = (-(R/2+2*R),  R/2+2*R) #  左上
pD = (-(R/2+2*R),-(R/2+2*R)) #  左下
#
# combine these four poins to make LineString()
```

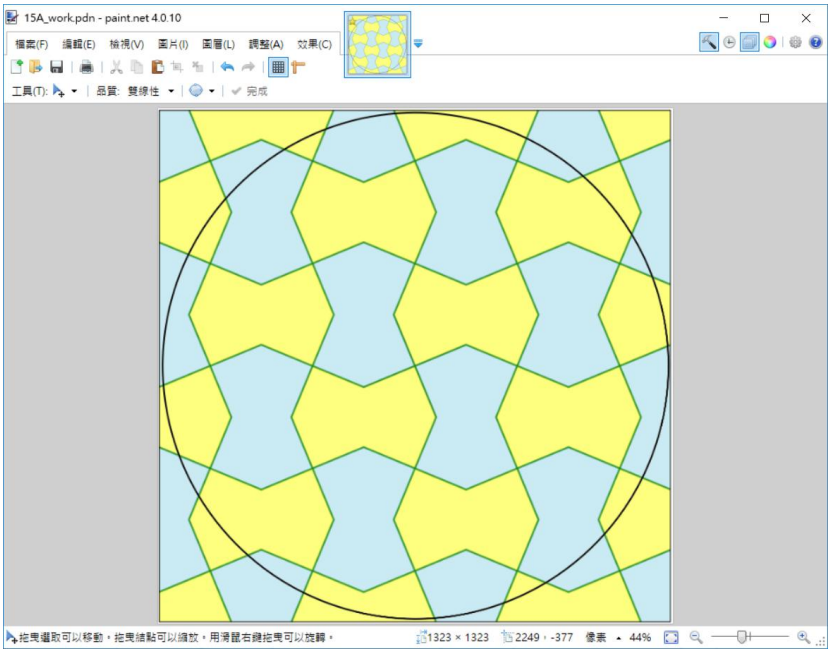
```
square = LineString([pA, pB, pC, pD, pA])
xs, ys = square.xy
ax.plot(xs, ys, alpha=1.0, color='blue', linewidth=1.5, solid_capstyle='round', zorder=30)
#
ax.set_title('', fontsize=22)   #  設定子圖的標題
plot.savefig("15.png", format="png", dpi=300)
#
print ('Done')
```

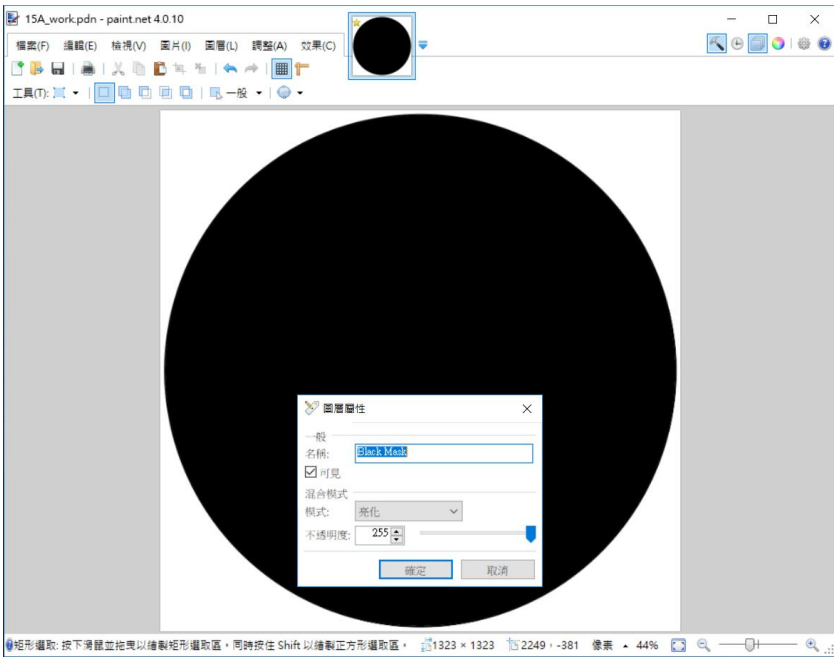
A16-如何製作杯墊? (How to make a coaster?)

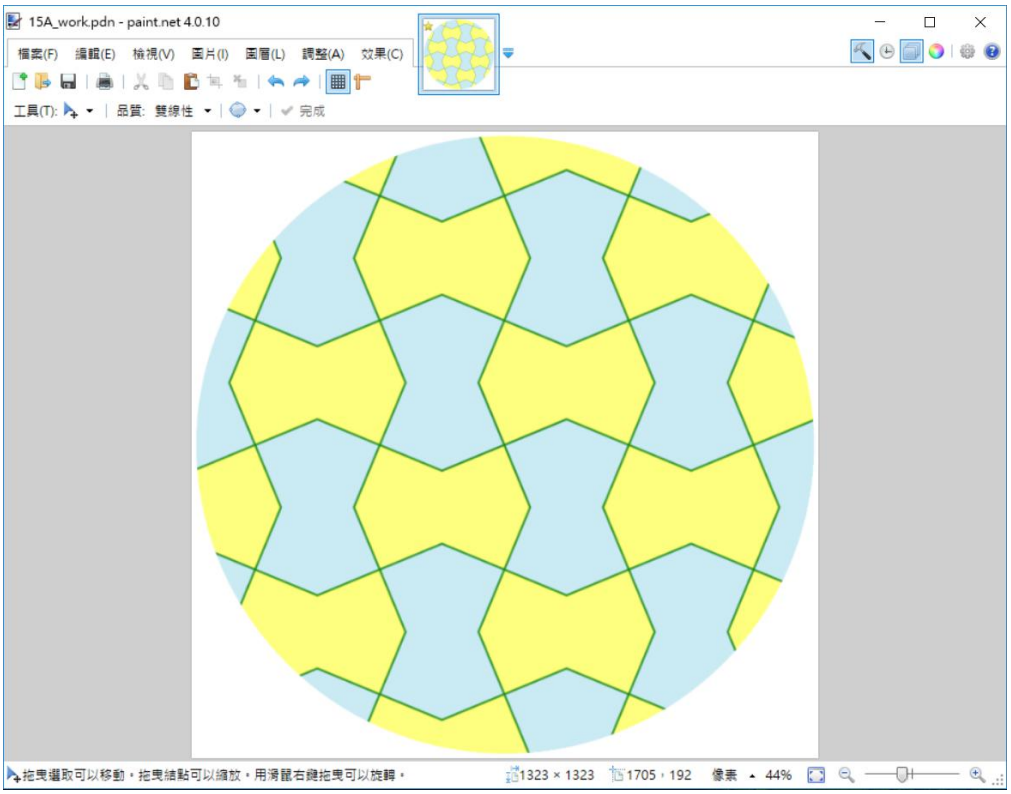
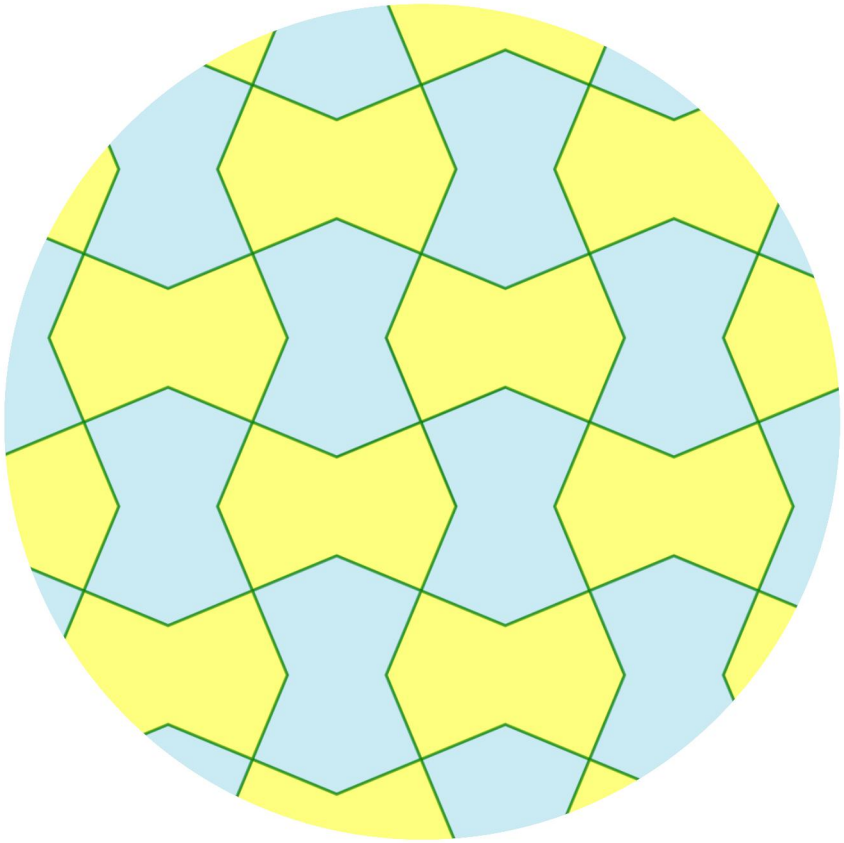
No program code in here
#
How to make coaster?
#
1). 15_1.png 圖檔用小畫家填色 or 用 Paint.Net 填色
2). 15_2.png 填色完成
3). 15_3.png 11.2cm 的空心黑圓
4). 啟動 Paint.Net
5). 開啟 15_2 圖檔 (如圖 paint.Net-1.png)
6). 開啟 15_3 圖檔 (如圖 paint.Net-2.png)
7). 15_3 圖檔在上層, 15_2 圖檔在下層 (如圖 paint.Net-3.png)
8). 15_3 圖檔填滿黑色, 圖層模式設為暗化 (如圖 paint.Net-4.png)
9). 使用暗化遮罩後得到的圖形 (如圖 paint.Net-5.png)
A). 將圖形另存 PNG 圖檔 (如圖 paint.Net-6.png)
B). 將 PNG 圖檔送交廣告公司製作杯墊 10 個以內, 每個 60 元,
10 個以內, 每個 60 元, 10 個以上, 每個 50 元,
C). 亞企廣告社 07-731-9959 (高雄 鳥松)




15A_work.pdn - paint.net 4.0.10
檔案(F) 編輯(E) 檢視(V) 圖片(I) 圖層(L) 調整(A) 效果(C)
工具(T): 品質: 雙線性 完成
拖曳選取可以移動。拖曳結點可以縮放。用滑鼠右鍵拖曳可以旋轉。
1323 × 1323
2251 , -390
像素
44%


15A_work.pdn - paint.net 4.0.10
檔案(F) 編輯(E) 檢視(V) 圖片(I) 圖層(L) 調整(A) 效果(C)
工具(T): 品質: 雙線性 完成
拖曳選取可以移動。拖曳結點可以縮放。用滑鼠右鍵拖曳可以旋轉。
1323 × 1323
2249 , -377
像素
44%


15A_work.pdn - paint.net 4.0.10
檔案(F) 編輯(E) 檢視(V) 圖片(I) 圖層(L) 調整(A) 效果(C)
工具(T): 一般
圖層屬性
一般
名稱: Black Mask
可見
混合模式
模式: 亮化
不透明度: 255
確定
取消
矩形選取: 按下滑鼠並拖曳以繪製矩形選取區。同時按住 Shift 以繪製正方形選取區。
1323 × 1323
2249 , -381
像素
44%


15A_work.pdn - paint.net 4.0.10
檔案(F) 編輯(E) 檢視(V) 圖片(I) 圖層(L) 調整(A) 效果(C)
工具(T): 品質: 雙線性 完成
拖曳選取可以移動。拖曳節點可以縮放。用滑鼠右鍵拖曳可以旋轉。
1323 × 1323
1705 , 192
像素
44%